Software Requirements Specification

TECHNICAL NEWS HUB

Submitted by:
Karan Vasudeva (24110001)
Ekamveer Singh (24110004)
Matisha Kansal (24110034)
Branch: CSE (AI)
Year: Second Year

Table of Contents

1. **Introduction**
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Definitions, Acronyms, and Abbreviations
    - 1.4 References
    - 1.5 Overview

2. **Overall Description**
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Characteristics
    - 2.4 Constraints
    - 2.5 Assumptions and Dependencies

3. **Specific Requirements**
    - 3.1 External Interface Requirements
        - 3.1.1 User Interface
        - 3.1.2 Hardware Interface
        - 3.1.3 Software Interface
    - 3.2 Functional Requirements
    - 3.3 Design Constraints
    - 3.4 Software System Attributes
    - 3.5 Other Requirements

4. **Verification and Validation Plan**
    - 4.1 Functional Requirement Verifications
    - 4.2 Quality Attribute Validation
    - 4.3 User Acceptance
    - 4.4 Stakeholder Review

1. Introduction

1.1 Purpose

This document provides a comprehensive Software Requirements Specification (SRS) for the **Technical News Hub System**, a scalable web-based platform designed to aggregate, classify, and present domain-specific technical news in a structured and intelligent manner.

The purpose of this document is to:

- Define complete system behavior before implementation
- Serve as a reference for development and testing
- Act as a contract between stakeholders and developers
- Provide traceability for validation and verification
- Support future scalability and enhancement

This SRS follows IEEE standards and is structured to meet academic requirements for the course project.

1.2 Scope

The Technical News Hub is a centralized technical news aggregation and classification system that integrates with third-party News APIs to collect real-time technical articles. The system processes incoming data, performs domain classification, stores structured records in a database, and presents filtered results to users through a responsive web interface.

The system addresses the following real-world problems:

- Information overload across multiple platforms
- Lack of domain-specific technical categorization
- Difficulty tracking emerging trends
- Scattered and unstructured content

System Objectives

1. Provide centralized technical updates

2. Automatically classify articles into domains
3. Enable structured search and filtering
4. Support scalable architecture

1.3 Definitions, Acronyms, and Abbreviations

API – Application Programming Interface
REST – Representational State Transfer
CRUD – Create, Read, Update, Delete
UI – User Interface
DB – Database
JWT – JSON Web Token
CI/CD – Continuous Integration / Continuous Deployment

1.4 References

- IEEE 830-1998 Software Requirements Specification
- IEEE 29148 Systems and Software Engineering
- React.js Documentation
- Node.js Documentation
- MongoDB Official Documentation
- NewsAPI Documentation

1.5 Overview

This document is organized into the following sections:

- Overall system description
- Detailed functional requirements
- Non-functional requirements
- External interfaces
- System attributes
- Testing strategies

---

2. Overall Description

2.1 Product Perspective

The **Technical News Hub** is a web-based news aggregation platform designed to collect, process, and present domain-specific technical news articles to users in a structured and searchable format. The system follows a **three-tier architecture**, ensuring separation of concerns, maintainability, and scalability.

The three architectural layers are described below:

2.1.1 Presentation Layer (Client Tier – React.js)

The Presentation Layer is responsible for user interaction and interface rendering. It is implemented using React.js and follows a component-based architecture.

This layer shall:

- Render article listings, detailed article views, and category pages.
- Provide user interfaces for search, filtering, and sorting.
- Maintain client-side state to ensure efficient rendering.
- Communicate with the backend using RESTful API calls.
- Ensure responsive design for compatibility across devices.

The Presentation Layer does not directly access the database and relies entirely on the Application Layer for data retrieval and updates.

2.1.2 Application Layer (Business Logic Tier – Node.js & Express.js)

The Application Layer acts as the core processing unit of the system. It manages business logic, request validation, and communication between the frontend and database.

Its primary responsibilities include:

API Management

- Exposing RESTful endpoints for article retrieval, search operations, and administrative tasks.
- Validating incoming requests and ensuring structured responses.

News Aggregation Handling

- Fetching news data from third-party APIs using secure API keys.
- Managing scheduled data retrieval processes.
- Handling API rate limits and error responses.

Data Processing

- Cleaning and normalizing incoming article data.
- Removing duplicate entries.
- Converting raw API responses into structured JSON format.
- Ensuring data consistency before storage.

Categorization Logic

- Assigning articles to predefined technical domains using rule-based keyword matching.
- Supporting future extensibility for machine learning-based classification.

Search Query Execution

- Processing keyword-based search requests.
- Supporting category-based filtering.
- Enabling date-based sorting of articles.
- Optimizing query performance through database indexing.

2.1.3 Data Layer (Persistence Tier – MongoDB)

The Data Layer provides persistent storage and manages structured data using MongoDB.

The database stores the following entities:

Articles

- Unique identifier
- Title
- Author
- Description
- Publication date
- Source information
- URL
- Category reference

Categories

- Predefined technical domains (e.g., AI, Cybersecurity, Cloud Computing).
- Category identifiers used for classification.

User Metadata

- Saved articles
- User preferences (if personalization is enabled)
- Non-sensitive interaction data

System Logs

- API request logs
- Error logs
- System activity records

Indexing mechanisms are implemented to improve search performance and reduce query latency.

2.1.4 External System Integration

The system integrates with third-party News APIs to retrieve real-time technical news content. API authentication is handled securely using stored credentials. The system depends on the availability and reliability of these external services.

2.2 Product Functions

The Technical News Hub shall perform the following major functions:

2.2.1 News Aggregation

The system shall periodically retrieve technical news articles from external APIs. Retrieved data shall be validated and processed before storage.

2.2.2 Data Normalization

The system shall clean and standardize incoming article data by:

- Removing incomplete or malformed entries
- Formatting timestamps
- Ensuring consistent field structures

2.2.3 Domain Classification

The system shall categorize articles into predefined technical domains using rule-based keyword matching. The architecture shall support future implementation of automated machine learning-based classification models.

2.2.4 Search and Filtering

The system shall allow users to retrieve articles using:

- Keyword-based search
- Domain-based filtering
- Date-based sorting (ascending or descending)
- Combined search criteria

Search results shall be displayed in a structured and paginated format.

2.2.5 Administrative Control

The system shall provide administrative functionality for authorized users, including:

- Creating, updating, and deleting categories
- Managing API configuration settings
- Monitoring system logs
- Viewing system activity metrics

Administrative access shall require authentication.

2.3 User Characteristics

2.3.1 General Users

General users include students, researchers, and professionals who seek structured technical news updates. They are expected to have basic familiarity with web applications but require no technical expertise.

2.3.2 Administrators

Administrators are technically proficient users responsible for system configuration, API management, and performance monitoring. They possess elevated privileges within the system.

2.4 Constraints

The system operates under the following constraints:

- API rate limits imposed by third-party news providers.
- Dependency on the uptime and availability of external APIs.
- Hosting server resource limitations (CPU, memory, storage).
- Database scalability considerations as article volume increases.
- Security requirements for protecting API keys and server configurations.

2.5 Assumptions and Dependencies

The system design assumes:

- Reliable internet connectivity for both users and server.
- Continued availability of third-party news APIs.
- MongoDB server accessibility and stability.
- Secure hosting environment with appropriate runtime support for Node.js.
- Proper configuration of authentication and security mechanisms.

---

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface

The system shall provide a web-based graphical user interface accessible through modern web browsers.

The user interface shall include:

Homepage Displaying Categorized News
The homepage shall present news articles organized under predefined technical domains. Articles shall be displayed in a structured format including title, short description, publication date, and source information.

Search Bar
A search input field shall allow users to enter keywords to retrieve relevant articles. The system shall dynamically display search results based on user queries.

Filter Dropdown
A filter mechanism shall allow users to select specific technical domains. The system shall update displayed results according to the selected category.

Responsive Layout
The interface shall adapt to various screen sizes including desktops, tablets, and mobile devices. Layout elements shall automatically adjust to maintain usability and readability.

Error Notifications
The system shall display clear and concise error messages in cases such as:

- Failed API requests
- No search results found
- Server connectivity issues

The overall user interface shall be intuitive, minimalistic, and user-friendly to ensure ease of navigation without requiring technical expertise.

3.1.2 Hardware Interface

The system shall operate on standard consumer devices without requiring specialized hardware.

Supported platforms include:

- Desktop and laptop computers running modern web browsers

- Smartphones with web browser support
- Tablet devices with internet connectivity

The system does not require dedicated hardware components beyond standard internet-enabled devices.

3.1.3 Software Interface

The system shall interact with the following software components:

NewsAPI (Third-Party News Service)
The Application Layer shall communicate with external News APIs using secure API keys to retrieve technical news articles in JSON format.

MongoDB Database
The system shall use MongoDB for persistent storage of articles, categories, user metadata, and logs. Communication between the backend and database shall occur through appropriate database drivers.

Node.js Runtime Environment
The backend shall operate within a Node.js runtime environment to execute server-side logic and API handling.

Express Middleware
Express.js middleware shall manage routing, request handling, validation, and security mechanisms within the backend application.

3.2 Functional Requirements

(Structured with formal numbering)

News Aggregation Module

REQ-1: The system shall fetch technical news articles from external APIs at defined intervals.
REQ-2: The system shall validate API responses before processing.
REQ-3: The system shall store validated articles in the database.

Categorization Module

REQ-4: The system shall classify articles into predefined domains.
REQ-5: Each article shall belong to at least one category.
REQ-6: Categories shall be editable by admin.

Search Module

REQ-7: The system shall allow users to search articles using keywords present in the title or description.
REQ-8: The system shall support filtering by category.
REQ-9: The system shall support sorting by publication date.

Display Module

REQ-10: The system shall display title, summary, source, and timestamp.
REQ-11: The system shall provide external link redirection.

3.3 Design Constraints

- RESTful API architecture mandatory
- Backend implemented in Node.js
- Frontend implemented in React.js
- Database implemented in MongoDB
- MVC design pattern recommended

3.4 Software System Attributes

Reliability

The system shall handle API failures and unexpected errors gracefully by providing appropriate fallback responses and user-friendly error notifications. It shall ensure data integrity and prevent system crashes during external service interruptions.

Availability

The system shall target a minimum uptime of 99%, subject to hosting and third-party API availability. Scheduled maintenance periods shall be minimized to reduce service disruption.

Security

API keys shall be securely stored using environment variables and shall not be exposed in client-side code. The system shall implement input validation and sanitation mechanisms to prevent malicious inputs. Protective measures shall be in place to mitigate common web vulnerabilities such as Cross-Site Scripting (XSS) and injection attacks.

Performance

The system shall respond to user requests within 3 seconds under normal operating conditions and moderate user load. It shall efficiently handle concurrent user requests without significant degradation in response time.

Scalability

The system shall support scaling to handle increased user traffic, enabling the addition of server instances to handle increased traffic and data volume.

Maintainability

The system codebase shall follow modular design principles and include adequate documentation to facilitate debugging, enhancement, and future feature expansion.

3.5 Other Requirements

- **Logging Mechanism**
  The system shall implement a logging mechanism to record system events, API requests, errors, and administrative actions. Logs shall assist in monitoring, debugging, and performance evaluation.

- **Error Tracking**
  The system shall capture and store runtime errors and exception details to facilitate troubleshooting and system reliability improvement. Critical errors shall be logged with timestamps and relevant context information.

- **Future AI Recommendation Module**
  The system architecture shall allow future integration of an AI-based recommendation module capable of suggesting articles based on user preferences and interaction history.

- **Analytics Dashboard**
  The system shall support the future development of an administrative analytics dashboard to display metrics such as article trends, API usage statistics, and user engagement data.

---

4. Verification and Validation Plan

The following activities will be conducted to ensure that the Technical News Hub system satisfies the requirements defined in Section 3.

4.1 Functional Requirement Verification

The system will be evaluated to confirm that all primary functional requirements are correctly implemented, including:

- Successful retrieval of technical news from external News APIs.
- Proper categorization of articles into predefined domains.
- Accurate keyword-based search functionality.
- Correct filtering and sorting of news articles.

Each feature will be tested using controlled input scenarios and verified against expected outputs.

4.2 Quality Attribute Validation

The system will be validated against defined non-functional requirements:

- **Performance:** Ensure user requests are processed within the target response time (≤ 3 seconds under normal conditions).
- **Security:** Confirm API keys are securely stored and not exposed in the frontend.
- **Reliability:** Verify appropriate handling of API failures and invalid inputs.

4.3 User Acceptance

A final walkthrough of the web interface will be conducted to confirm that users can:

- Browse categorized technical news.
- Perform keyword searches successfully.
- Navigate to external article sources without errors.

4.4 Stakeholder Review

The completed system will be reviewed by faculty stakeholders to ensure it meets project objectives, architectural expectations, and academic standards outlined in this document.